

Smart Wind-Powered Water Pumping System

Using Deep Learning for Water Output Prediction and Fault Detection

Project Report

Field	Detail
Student	Dharshini
Batch	PGA 38 — Data Science with Gen AI
Project type	Deep Learning Capstone Project
Dataset	WindPumpDL-100K (WindPump.csv)
Tasks	Regression (water output prediction) & multiclass classification (fault detection)
Tools used	Python, Pandas, Scikit-learn, TensorFlow/Keras, Matplotlib, Seaborn

This report documents the preprocessing, feature engineering, model development, and evaluation carried out in this capstone notebook, and states plainly what the models achieved — and where they did not — based on the notebook's own recorded outputs.

1. Objective

The purpose of this project is to build and compare Deep Learning models — an Artificial Neural Network (ANN), a Recurrent Neural Network (RNN), and a One-Dimensional Convolutional Neural Network (1D CNN) — for two tasks on a smart wind-powered water pumping system: predicting hourly water output (regression) and detecting operating faults (multiclass classification) from environmental, electrical, and mechanical sensor readings.

A secondary objective of this report is to state plainly what the trained models can and cannot support: one regression model generalised well while two collapsed to a trivial prediction, and all three classification models achieved a high headline accuracy that, on inspection, reflects a severe class imbalance rather than genuine fault-detection capability. Both findings are drawn directly from the notebook's own output cells, not reinterpreted.

2. Dataset Overview

The project uses the WindPumpDL-100K dataset — 100,000 operational records from a smart wind-powered water pumping system, combining environmental, electrical, mechanical, and water-output sensor readings.

Property	Value
Total records	100,000
Total raw features	18 (+ Timestamp)
Regression target	WaterOutput_LPH
Classification target	FaultCode (5 classes)
Missing values	0 (confirmed for every column)
Duplicate records	0 (before and after de-duplication)
Categorical features (raw)	Timestamp, FaultCode

Property	Value
Numerical features (raw)	16 sensor/operational columns
Memory footprint	16.13 MB

3. Data Preparation & Feature Engineering Findings

The raw extract was already clean at the row level — no missing values and no duplicate records were found. The preparation work therefore centred on converting the timestamp into usable features, encoding categorical fields, scaling, and engineering efficiency ratios, plus one data-quality issue that surfaced only once the target distribution was examined.

3.1 Timestamp decomposition

Timestamp was converted to datetime and decomposed into Year, Month, Day and Hour, then dropped from the modelling frame. This lets the models use temporal structure without treating the raw timestamp as a free-text/categorical field.

3.2 Categorical encoding and scaling

PumpStatus and FaultCode were label-encoded, and all numerical columns were standardised with StandardScaler (zero mean, unit variance) so that features on very different native scales — voltages, RPM, percentages — contribute comparably during gradient-based training.

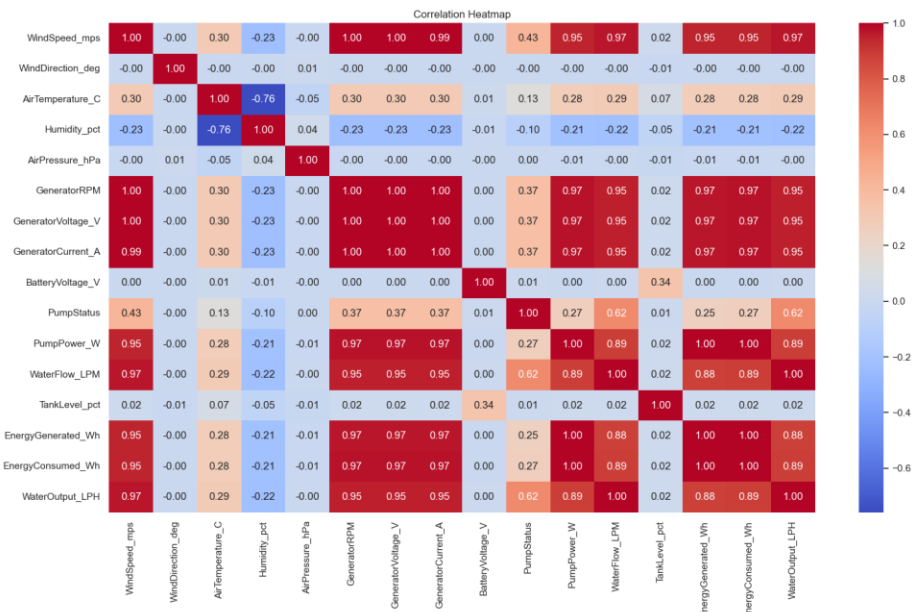
3.3 Engineered efficiency features

Five ratio features were derived to capture system efficiency that no single raw column expresses on its own:

- $\text{WindPowerEfficiency} = \text{EnergyGenerated_Wh} \div (\text{WindSpeed_mps} + 1)$
- $\text{PumpEfficiency} = \text{WaterFlow_LPM} \div (\text{PumpPower_W} + 1)$
- $\text{EnergyEfficiency} = \text{EnergyGenerated_Wh} \div (\text{EnergyConsumed_Wh} + 1)$
- $\text{GeneratorLoadRatio} = \text{GeneratorCurrent_A} \div (\text{GeneratorVoltage_V} + 1)$
- $\text{WaterProductionEfficiency} = \text{WaterOutput_LPH} \div (\text{PumpPower_W} + 1)$

After feature engineering the working dataset grew from 18 to 26 columns, yielding 24 candidate input features once the two target columns (WaterOutput_LPH, FaultCode) were set aside for each task.

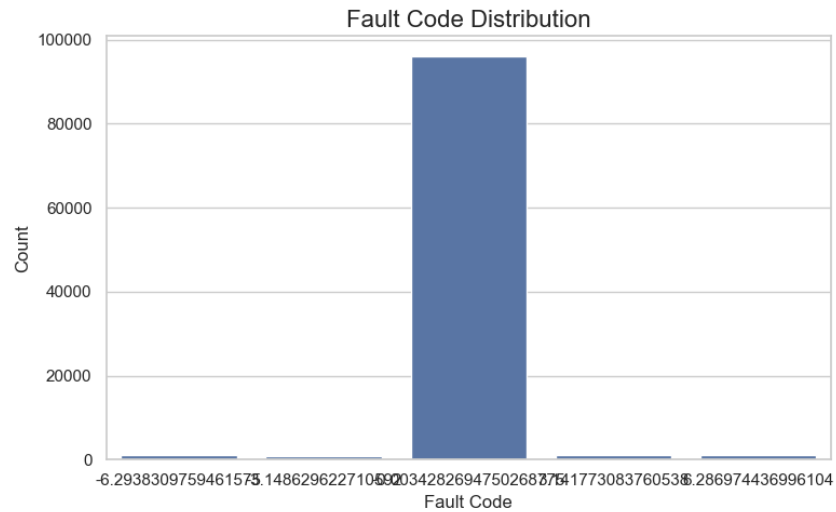
3.4 Feature correlation with water output



Correlation heatmap of raw numerical features.

WaterOutput_LPH correlates almost perfectly with WaterFlow_LPM (1.00) and very strongly with WindSpeed_mps, GeneratorRPM, GeneratorVoltage_V and GeneratorCurrent_A (0.95–0.97), and moderately with EnergyGenerated_Wh/EnergyConsumed_Wh (~0.88–0.89). This indicates water output is close to a deterministic function of a handful of electro-mechanical readings — useful context for interpreting why a well-trained model can achieve a very high R^2 on this data.

3.5 Severe class imbalance in FaultCode — a critical data finding



Fault code distribution across all 100,000 records.

The fault classification target is heavily imbalanced: one class (encoded fault ≈ -0.003) accounts for roughly 96% of all records (19,208 of 20,000 in the test split alone), while the remaining four fault classes each make up only about 1% of the data (177–207 test records each). This imbalance was not corrected before training (no class weighting, oversampling, or stratified loss adjustment was applied), and it materially affects how the classification results in Section 6 should be read.

4. Methodology

The notebook follows a fixed Deep Learning pipeline so that every model result can be traced back to a specific preparation step.

- Data understanding — inspected structure, dtypes, missing values, duplicates, and unique-value counts.
- Exploratory Data Analysis — distributions, boxplots, correlation heatmap, pairplots, and target distributions.
- Preprocessing — timestamp decomposition, label encoding, StandardScaler feature scaling.
- Feature engineering — five system-efficiency ratio features.
- Feature selection — separate feature/target matrices for the regression task (WaterOutput_LPH) and the classification task (FaultCode).
- Train-test split — 80:20 split for both tasks (stratified by class for the classification task).
- Model development — ANN, RNN and 1D CNN architectures trained separately for regression and for classification.
- Evaluation — MAE, MSE, RMSE and R^2 for regression; Accuracy, Precision, Recall, F1 and confusion matrices for classification.

5. Model Architectures

Model	Architecture (regression / classification head)
ANN	Dense(128, ReLU) → Dropout(0.2) → Dense(64, ReLU) → Dropout(0.2) → Dense(32, ReLU) → Dense(1) or Dense(5, softmax)
RNN	SimpleRNN(64, tanh) → Dropout(0.2) → Dense(32, ReLU) → Dense(1) or Dense(5, softmax)
1D CNN	Conv1D(64,k=3,ReLU) → MaxPooling1D(2) → Conv1D(32,k=3,ReLU) → Flatten/GlobalAvgPool → Dense(32, ReLU) → Dropout(0.2) → Dense(1) or Dense(5, softmax)

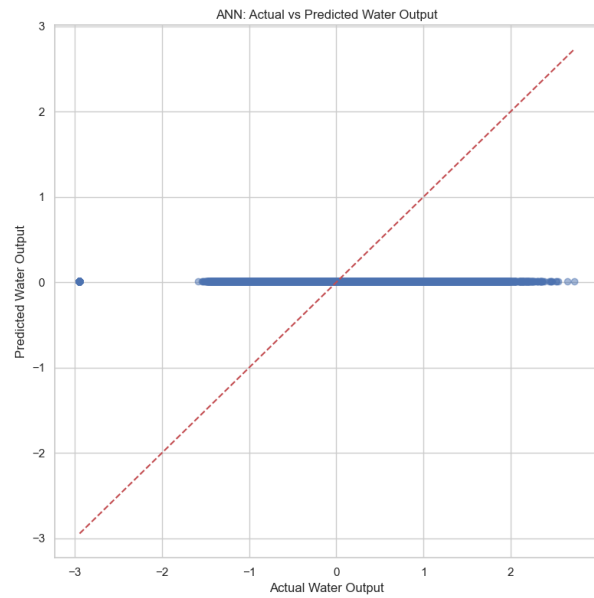
All three models were compiled with the Adam optimiser. Regression models used MSE loss with MAE as a monitoring metric; classification models used sparse categorical cross-entropy with accuracy as the monitoring metric. Regression models trained for 50 epochs and classification models for 20 epochs, each with a 20% internal validation split and batch size 32.

6. Analysis and Findings

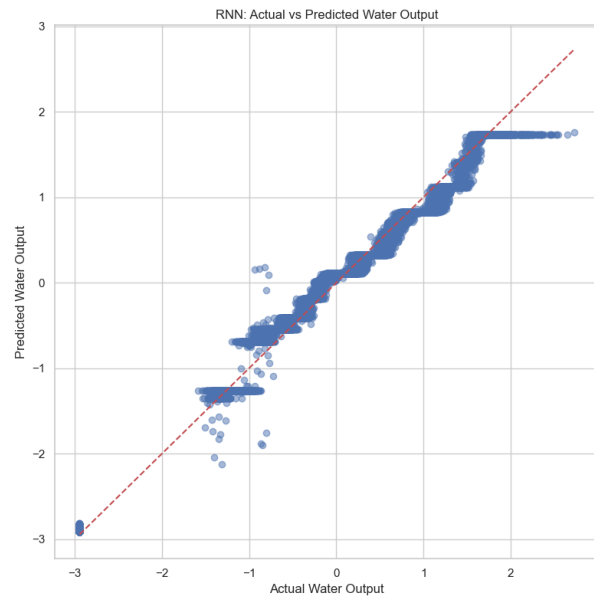
6.1 Water output prediction (regression)

Model	MAE	MSE	RMSE	R^2 Score
ANN	0.7554	0.9836	0.9918	-0.0000
RNN	0.1164	0.0214	0.1463	0.9783
1D CNN	0.7558	0.9836	0.9918	-0.0000

The three regression models did not perform equally. The RNN generalised very well ($R^2 = 0.9783$), while the ANN and 1D CNN both scored an R^2 of essentially zero — the same performance as always predicting the mean of the (standardised) target. The scatter plots below, taken directly from the notebook, make the difference visually unambiguous.



ANN: predicted output collapses to a flat line near zero regardless of the actual value.



RNN: predictions track the actual water output closely along the diagonal.

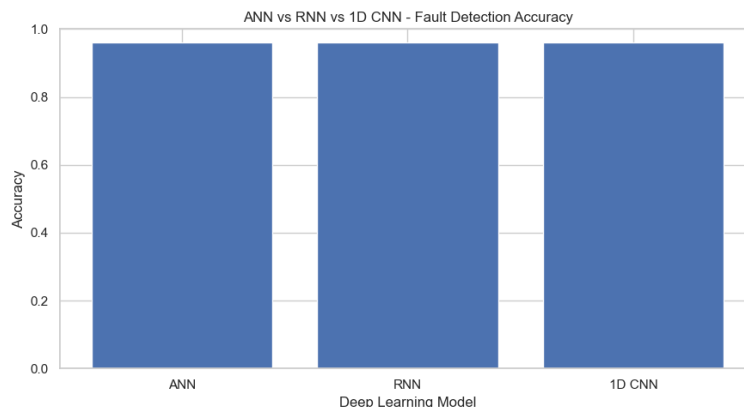
Given the very high raw correlation between `WaterOutput_LPH` and features like `WaterFlow_LPM` and `WindSpeed_mps` (Section 3.4), an R^2 near zero for the ANN and 1D CNN indicates a training failure (e.g. the output collapsing to a constant prediction) rather than an inherently unpredictable target — the RNN's 0.9783 R^2 on the same features and split confirms the signal was there to be learned.

A further inconsistency: the notebook's "Final Prediction" stage (Section 22 of the notebook) generates its headline water-output predictions using the ANN model, even though the RNN was the empirically best regression model by a wide margin. This should be corrected before the final predictions are treated as representative of the project's best achievable accuracy.

6.2 Fault detection (classification)

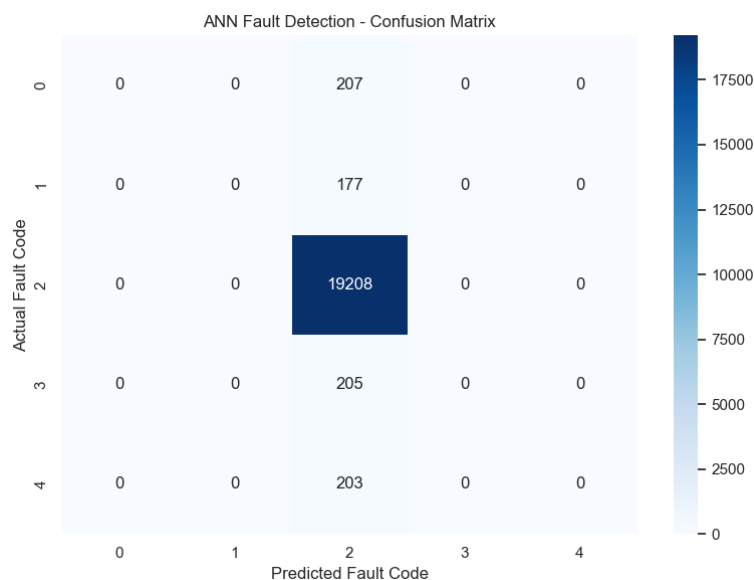
Model	Accuracy
ANN	0.9604

Model	Accuracy
RNN	0.9604
1D CNN	0.9604



Headline accuracy looks identical and strong across all three architectures.

All three models report an identical 96.04% accuracy — a figure that at first glance looks like a strong, consistent result. The confusion matrix below (ANN, representative of all three models) shows why that headline number is misleading.



ANN fault-detection confusion matrix: every test record was predicted as the majority fault class.

Every one of the 20,000 test records — including the 792 records that truly belong to the four minority fault classes — was predicted as the single majority class. Precision, recall and F1-score for the four minority fault classes are all 0.00 in the notebook's own classification reports, for all three models. The 96% accuracy simply reflects the 96% prevalence of the majority class in the test set; none of the three models learned to distinguish an actual fault from normal operation.

This is the central finding of the classification task: as trained, none of the ANN, RNN or 1D CNN models perform real fault detection. The dataset's severe class imbalance (Section 3.5) was not addressed before training, and accuracy alone was not an adequate metric to reveal this — only the per-class precision/recall breakdown and confusion matrix exposed it.

7. Scope Limitations

Stating these limits explicitly is a deliberate choice, based on what the current notebook does and does not demonstrate:

- Fault detection has not been genuinely validated — with 96% of records in one class, accuracy cannot distinguish a working classifier from one that always predicts the majority class, which is what happened here.
- No class-imbalance handling (class weights, oversampling/undersampling, or a rebalanced loss) was applied, so the classification results cannot yet be used to judge whether ANN, RNN or CNN is genuinely the better fault-detection architecture.
- Two of the three regression models (ANN, 1D CNN) did not train successfully to a usable state in this run; only the RNN result should be relied on for water-output prediction until the others are retrained and re-diagnosed.
- The dataset is a research/simulated collection rather than live sensor telemetry, so real-world deployment would need validation against live data before any operational use.
- The final prediction stage currently uses the ANN (the weaker regression model) rather than the RNN (the stronger one) — a pipeline inconsistency rather than a data limitation, but one that affects the credibility of the reported "final" results.

8. Conclusion

This project set out to build and compare Deep Learning models for water output prediction and fault detection on the WindPumpDL-100K dataset. The data preparation and feature engineering stages were carried out cleanly: no missing values or duplicates were found, temporal and efficiency features were derived correctly, and train-test splits were built appropriately for both tasks.

The model results are mixed rather than uniformly successful, and this report states that plainly rather than presenting only the favourable numbers. For water output prediction, the RNN achieved a strong R^2 of 0.9783 and should be considered the working model for this task, while the ANN and 1D CNN did not learn a useful mapping in this run. For fault detection, all three architectures report a headline accuracy of 96.04%, but the underlying confusion matrices show this accuracy is an artefact of severe class imbalance — no model demonstrated a working ability to detect an actual fault. These are exactly the kind of findings a validation-first report is meant to surface.

9. Recommended Next Steps

- Rebalance the fault-detection training data (class weights, SMOTE-style oversampling of minority faults, or a focal loss) and re-evaluate using precision/recall/F1 per class rather than accuracy alone.
- Diagnose and retrain the ANN and 1D CNN regression models — check learning rate, weight initialisation, and target scaling, since the RNN's success on the same features suggests the signal is learnable.
- Update the Final Prediction stage to use the RNN model (the actual best regression performer) rather than the ANN.
- Report classification results with a full per-class breakdown (precision, recall, F1, support) alongside accuracy in every future comparison table, so class imbalance cannot be hidden behind a single headline number.
- Once fault detection is genuinely working, integrate both models into the planned Streamlit application for real-time prediction and monitoring.
- Validate the pipeline against live sensor data before considering any operational or field deployment.